

# AI-Enhanced Deep Packet Inspection Firewall

*Machine Learning-Powered Network Security System*

Python · Scapy · NetfilterQueue · scikit-learn · Flask · JA3 TLS

|              |                                                             |
|--------------|-------------------------------------------------------------|
| Project Type | Cybersecurity / Network Security System                     |
| Domain       | Deep Packet Inspection, Intrusion Detection, ML in Security |
| Language     | Python 3.x                                                  |
| ML Models    | Random Forest Classifier, Logistic Regression               |
| Dataset      | CSE-CIC-IDS2018 (Canadian Institute for Cybersecurity)      |
| Accuracy     | Random Forest: 98.7%   Logistic Regression: 96.1%           |
| Status       | Completed — Production-ready prototype                      |

## 1. Project Overview

The AI-Enhanced Deep Packet Inspection Firewall is a comprehensive network security system that combines traditional firewall mechanisms with Machine Learning-based threat detection and Deep Packet Inspection (DPI) techniques. The system is designed to monitor, filter, and analyze network traffic in real time, identifying malicious packets, suspicious requests, and abnormal encrypted traffic patterns with a high degree of accuracy.

Unlike conventional firewalls that rely solely on static rule sets, this system employs trained ML models alongside regex-based payload scanning and JA3 TLS fingerprinting to detect both known and previously unseen attack vectors. The result is a multi-layered defence architecture capable of adapting to evolving cybersecurity threats.

### Core Objective

To develop an intelligent, AI-powered firewall that captures live network packets, inspects their content, applies rule-based and machine-learning-driven classifications, and takes automated action — blocking malicious traffic while permitting legitimate communication — in real time.

## 2. Key Features

### 2.1 Real-Time Packet Monitoring

The system captures live network packets directly from the system's network interface using Scapy, a powerful Python-based packet manipulation library. All incoming and outgoing traffic is monitored continuously without interruption.

- Captures raw packets from the active network interface
- Monitors both incoming and outgoing traffic simultaneously
- Uses packet sniffing techniques for deep traffic analysis
- Redirects packets via NetfilterQueue for processing before verdict delivery

## 2.2 Deep Packet Inspection (DPI)

The DPI engine inspects packet payloads beyond basic header information. It uses a library of regular expressions to detect known malicious patterns in the data portion of captured packets, providing a second layer of defence against application-layer attacks.

| Attack Type                | Example Pattern          | Description                   |
|----------------------------|--------------------------|-------------------------------|
| SQL Injection              | ' OR 1=1 --              | Classic authentication bypass |
| SQL Drop                   | DROP TABLE               | Destructive database command  |
| SQL Union                  | UNION SELECT *           | Data exfiltration via SQL     |
| Cross-Site Scripting (XSS) | <script>                 | Injected JavaScript execution |
| Path Traversal             | ../../../../etc/passwd   | File system access attempt    |
| Suspicious URLs            | Malicious URL parameters | Encoded attack parameters     |

## 2.3 AI-Based Threat Detection

At the heart of the system is an ensemble of two Machine Learning models trained on the CSE-CIC-IDS2018 network intrusion dataset. The models analyse over 70 statistical flow features extracted from each packet, classifying traffic as benign or malicious. The ensemble approach reduces false positives compared with any single model.

### Models Used

- Random Forest Classifier — Accuracy: 98.7%, Precision: 97.2%
- Logistic Regression — Accuracy: 96.1%, Precision: 94.8%

### Features Extracted (70+ metrics)

- Packet size statistics (min, max, mean, standard deviation)
- Inter-arrival times (IAT mean, std, max, min)
- Destination / source port numbers
- Protocol type (TCP, UDP, ICMP)
- TCP flags (SYN, ACK, FIN, RST, PSH, URG)
- Flow duration and total bytes transferred
- Window sizes and payload entropy
- Traffic frequency and burst patterns

Why an Ensemble?

Using both Random Forest and Logistic Regression provides complementary strengths: Random Forest excels at capturing non-linear decision boundaries and handling high-dimensional data, while Logistic Regression offers interpretable, fast classification for linearly separable patterns. Any packet flagged by either model is quarantined for further inspection.

## 2.4 Rule-Based Firewall Filtering

In addition to ML inference, the firewall enforces configurable rule-based policies for immediate, deterministic blocking. These rules are applied before expensive ML inference, reducing computational overhead and latency.

- IP address blacklisting and whitelisting
- Port-based blocking (source and destination)
- Protocol-level filtering (TCP, UDP, ICMP)
- Traffic redirection and rate limiting
- Seamless integration with Linux iptables via NetfilterQueue

## 2.5 JA3 TLS Fingerprinting

A significant proportion of modern network traffic is encrypted via TLS/HTTPS, rendering payload inspection ineffective. The JA3 fingerprinting module addresses this by analysing the TLS ClientHello handshake metadata — cipher suites, TLS version, extensions, elliptic curves — to generate a unique 128-bit MD5 fingerprint for each TLS client.

- Detects suspicious TLS client behaviour without decrypting traffic
- Identifies known malware communication fingerprints
- Flags anomalous cipher suite combinations
- Works on all HTTPS and encrypted TCP streams

## 2.6 Logging and Traffic Analysis

Every suspicious packet and detected threat is logged with full metadata for forensic analysis, incident response, and monitoring.

| Logged Field     | Description                            |
|------------------|----------------------------------------|
| Source IP        | Origin IP address of the packet        |
| Destination IP   | Target IP address                      |
| Port Numbers     | Source and destination ports           |
| Protocol Type    | TCP / UDP / ICMP / other               |
| Threat Status    | ALLOWED / BLOCKED / FLAGGED            |
| Timestamp        | ISO 8601 datetime of capture           |
| Payload Snippet  | First 64 bytes of payload (ASCII-safe) |
| Detection Method | Rule / DPI / RF-ML / LR-ML / JA3       |

### 3. System Architecture

The system is structured as a layered processing pipeline. Each packet passes through successive stages; early stages use low-cost operations (IP lookup, rule matching) while computationally intensive ML inference and JA3 analysis are applied only to packets that survive earlier filters.

#### 3.1 Processing Pipeline

| Stage               | Module            | Action                           | Technology                |
|---------------------|-------------------|----------------------------------|---------------------------|
| 1 — Capture         | Packet Sniffer    | Intercept raw packets            | Scapy                     |
| 2 — Queue           | NetfilterQueue    | Hold packet pending verdict      | NetfilterQueue + iptables |
| 3 — IP/Port Filter  | Rule Engine       | Block/pass by IP, port, protocol | Python dict lookup        |
| 4 — DPI Scan        | DPI Engine        | Regex payload analysis           | Python re module          |
| 5 — Feature Extract | Feature Extractor | Compute 70+ flow metrics         | Scapy + NumPy             |
| 6 — ML Inference    | ML Classifier     | RF + LR ensemble classification  | scikit-learn              |
| 7 — JA3 Check       | TLS Fingerprinter | Hash ClientHello fields          | Python hashlib            |
| 8 — Verdict         | Decision Engine   | ALLOW / BLOCK / LOG              | NetfilterQueue verdict    |

#### 3.2 Component Breakdown

##### basic\_firewall.py

Implements the foundational rule-based filtering layer. Parses IP headers and TCP/UDP ports, checks against a configurable block list, and delivers immediate verdicts without payload inspection.

##### dpi\_firewall.py

Extends the basic firewall with Deep Packet Inspection. Extracts the payload from each TCP/UDP packet and applies a comprehensive library of compiled regular expressions. Detected patterns are logged and the packet is dropped.

##### train\_firewall\_model.py

Handles dataset ingestion, preprocessing, feature engineering, model training, cross-validation, and serialisation of the trained Random Forest and Logistic Regression models. Outputs model files loaded at runtime by the main firewall process.

##### Flask API

Provides a RESTful API for external integration, enabling rule updates, threat log retrieval, and real-time statistics without restarting the firewall process.

### 4. Technology Stack

| Category         | Technology             | Purpose                                   |
|------------------|------------------------|-------------------------------------------|
| Language         | Python 3.x             | Core development language                 |
| Packet Capture   | Scapy                  | Low-level packet sniffing and crafting    |
| Queue Management | NetfilterQueue         | Kernel-to-userspace packet bridging       |
| ML Framework     | scikit-learn           | Model training, evaluation, serialisation |
| ML Model 1       | Random Forest          | Ensemble tree-based anomaly detection     |
| ML Model 2       | Logistic Regression    | Fast linear classification                |
| Pattern Matching | Python re (Regex)      | Payload signature scanning                |
| API Layer        | Flask                  | REST API for rule management              |
| TLS Analysis     | JA3 / TLS fingerprints | Encrypted traffic behaviour analysis      |
| Traffic Analysis | Wireshark              | Offline capture inspection and debug      |
| Firewall Backend | iptables               | Linux kernel firewall integration         |
| Dataset          | CSE-CIC-IDS2018        | Network intrusion training data           |

## 5. Challenges Faced

### 5.1 Real-Time Processing Performance

Processing every packet in user space introduces latency. The solution involved applying lightweight rules first (IP lookup, port filter) to reject obvious offenders cheaply, reserving the ML pipeline for ambiguous traffic only. Compiled regex patterns and pre-loaded model objects further reduced per-packet overhead.

### 5.2 Reducing False Positives

Initial single-model classifiers produced unacceptable false-positive rates for certain encrypted protocols. Switching to an ensemble approach — where both RF and LR must agree before blocking — significantly reduced false positives while maintaining high recall for true threats.

### 5.3 Encrypted HTTPS Traffic

Payload inspection is not viable for TLS-encrypted streams. The JA3 fingerprinting module was integrated to provide behavioural signals from the TLS handshake layer, allowing malware-associated clients to be flagged without compromising privacy by decrypting content.

### 5.4 Linux Kernel Integration

Bridging Python user-space code with Linux kernel firewall hooks via iptables and NetfilterQueue required careful handling of packet verdicts to avoid kernel queue starvation. Timeout and exception handling was added to ensure packets are always returned to the kernel, even in error conditions.

## 6. Learning Outcomes

---

1. Gained deep understanding of TCP/IP networking, packet structures, and protocol headers at the byte level.
2. Acquired hands-on experience with cybersecurity tools — Scapy, Wireshark, iptables — in a live environment.
3. Implemented Machine Learning in a real-time, latency-sensitive application rather than a static batch setting.
4. Understood the practical limitations of payload inspection against encrypted traffic and applied JA3 as a solution.
5. Designed and integrated a multi-layered security architecture balancing performance and thoroughness.
6. Worked with the CSE-CIC-IDS2018 dataset: data cleaning, feature engineering, class imbalance handling, and model evaluation.

## 7. Future Improvements

---

### 7.1 Web Dashboard

A real-time monitoring dashboard using React or Vue with WebSocket updates would allow security operators to visualise live traffic, threat counts, and model confidence scores without CLI access.

### 7.2 Deep Learning Models

Replacing or augmenting the classical ML models with LSTM or Transformer-based sequence models could improve detection of temporally correlated attack patterns such as slow-rate DDoS and stealthy port scans.

### 7.3 Threat Intelligence Integration

Automated feeds from threat intelligence providers (e.g., AbuseIPDB, Shodan) could enrich the rule engine with up-to-date IP blacklists and known malware C2 server fingerprints.

### 7.4 Distributed Deployment

Extending the architecture to support distributed sensor nodes — each forwarding aggregated flow statistics to a central ML inference server — would enable enterprise-scale deployment across large networks.

### 7.5 Performance Optimisation

Migrating the hot path (IP filter + DPI) to a C extension or using eBPF for in-kernel filtering would dramatically increase throughput for high-bandwidth environments (10 Gbps+).

## 8. Project Summary

| Aspect            | Detail                                              |
|-------------------|-----------------------------------------------------|
| Project Name      | AI-Enhanced Deep Packet Inspection Firewall         |
| Core Innovation   | Hybrid ML + DPI + JA3 multi-layer detection         |
| Models            | Random Forest (98.7%) + Logistic Regression (96.1%) |
| Detection Layers  | 3 — Rule-based, DPI/Regex, ML Ensemble + JA3        |
| Flow Features     | 70+ statistical metrics per flow                    |
| Encrypted Traffic | Handled via JA3 TLS fingerprinting                  |
| API               | Flask REST API for rule management                  |
| OS Integration    | Linux iptables + NetfilterQueue                     |
| Key Skills Shown  | Python, Networking, ML/AI, Cybersecurity, Linux     |

----- End of Document -----